

NOVA COMPONENTS

Diccionario de Datos Definitivo

Modelo operacional mínimo - 9 tablas

Masterclass End-to-End: Azure SQL + Blob Storage + Python + Power BI

Objetivo: documentar exactamente el modelo mínimo que se ha creado en Azure SQL. Solo se mantienen columnas que participan en relaciones, lógica de negocio o análisis posterior.

1. Resumen del modelo

Esquema	Tablas	Función
master	plant, customer, material, production_line	Datos maestros del negocio.
sales	sales_order, sales_order_line	Pedidos de clientes.
production	production_order, production_event	Fabricación y eventos productivos.
inventory	inventory_movement	Entradas y salidas de stock.

Flujo simplificado: Cliente -> Pedido -> Línea de pedido -> Stock -> Orden de producción -> Evento de producción -> Movimiento de inventario.

2. master.plant

Propósito: Representa las plantas o centros productivos de Nova Components.

Granularidad: Una fila por planta.

Columna	Tipo SQL	Clave	Explicación
plant_id	INT IDENTITY(1,1)	PK	Identificador interno automático de la planta.
plant_code	VARCHAR(10)	UNIQUE	Código corto y visible de la planta, por ejemplo P01.
plant_name	VARCHAR(100)	-	Nombre de la planta.
city	VARCHAR(100)	-	Ciudad donde se encuentra la planta.

Nota clave: Es una tabla base: material, production_line, sales_order e inventory_movement dependen de la planta.

3. master.customer

Propósito: Maestro de clientes que realizan pedidos.

Granularidad: Una fila por cliente.

Columna	Tipo SQL	Clave	Explicación
---------	----------	-------	-------------

customer_id	INT IDENTITY(1,1)	PK	Identificador interno automático del cliente.
customer_code	VARCHAR(20)	UNIQUE	Código único del cliente.
customer_name	VARCHAR(100)	-	Nombre comercial del cliente.

Nota clave: Se relaciona con sales.sales_order mediante customer_id.

4. master.material

Propósito: Define los productos/materiales gestionados por cada planta.

Granularidad: Una fila por material y planta.

Columna	Tipo SQL	Clave	Explicación
material_id	INT IDENTITY(1,1)	PK	Identificador interno del registro de material.
plant_id	INT	FK	Planta en la que este material está definido.
material_code	VARCHAR(30)	UK compuesta	Código del producto dentro de la planta.
material_name	VARCHAR(100)	-	Descripción del material.
product_family	VARCHAR(30)	-	Familia productiva usada para asignar una línea compatible.
unit_price	DECIMAL(12,2)	-	Precio de referencia del material.
safety_stock	DECIMAL(14,3)	-	Stock mínimo recomendado para esa planta.

Nota clave: UNIQUE (plant_id, material_code). El mismo material_code puede existir en distintas plantas con diferente safety_stock.

5. master.production_line

Propósito: Representa las líneas de producción disponibles en cada planta.

Granularidad: Una fila por línea de producción.

Columna	Tipo SQL	Clave	Explicación
production_line_id	INT IDENTITY(1,1)	PK	Identificador interno de la línea.
plant_id	INT	FK	Planta a la que pertenece la línea.
line_code	VARCHAR(20)	UK compuesta	Código de la línea dentro de la planta.
line_name	VARCHAR(100)	-	Nombre descriptivo de la línea.
product_family	VARCHAR(30)	-	Familia de materiales que puede fabricar.
daily_capacity	DECIMAL(14,3)	-	Capacidad máxima aproximada por día.

Nota clave: UNIQUE (plant_id, line_code). Python comprobará que material.product_family coincide con production_line.product_family.

6. sales.sales_order

Propósito: Cabecera de cada pedido realizado por un cliente.

Granularidad: Una fila por pedido.

Columna	Tipo SQL	Clave	Explicación
sales_order_id	BIGINT IDENTITY(1,1)	PK	Identificador interno del pedido.
order_number	VARCHAR(30)	UNIQUE	Número visible y único del pedido.
customer_id	INT	FK	Cliente que realiza el pedido.
plant_id	INT	FK	Planta responsable del pedido.
order_date	DATE	-	Fecha en la que se recibe el pedido.
requested_delivery_date	DATE	-	Fecha solicitada de entrega.
order_status	VARCHAR(30)	-	Estado general del pedido.

Nota clave: Estados recomendados: CREATED, IN_PROGRESS, PARTIALLY_DELIVERED, COMPLETED y CANCELLED.

7. sales.sales_order_line

Propósito: Detalle de los materiales y cantidades incluidos en cada pedido.

Granularidad: Una fila por material dentro de un pedido.

Columna	Tipo SQL	Clave	Explicación
sales_order_line_id	BIGINT IDENTITY(1,1)	PK	Identificador interno de la línea.
sales_order_id	BIGINT	FK	Pedido al que pertenece la línea.
material_id	INT	FK	Material solicitado.
ordered_quantity	DECIMAL(14,3)	-	Cantidad pedida.
delivered_quantity	DECIMAL(14,3)	-	Cantidad acumulada ya entregada.
unit_price	DECIMAL(12,2)	-	Precio aplicado al material en ese pedido.

Nota clave: Permite calcular Pending Quantity = ordered_quantity - delivered_quantity y Order Value = ordered_quantity * unit_price.

8. production.production_order

Propósito: Orden que indica qué material fabricar, en qué línea y en qué cantidad.

Granularidad: Una fila por orden de producción.

Columna	Tipo SQL	Clave	Explicación
production_order_id	BIGINT IDENTITY(1,1)	PK	Identificador interno de la orden.
material_id	INT	FK	Material que se va a fabricar.
production_line_id	INT	FK	Línea donde se realizará la fabricación.
planned_quantity	DECIMAL(14,3)	-	Cantidad planificada a fabricar.
order_status	VARCHAR(20)	-	Estado de la orden.

Nota clave: Estados recomendados: PLANNED, IN_PROGRESS, COMPLETED y CANCELLED.

9. production.production_event

Propósito: Registra eventos productivos periódicos generados por el simulador Python.

Granularidad: Una fila por evento productivo.

Columna	Tipo SQL	Clave	Explicación
---------	----------	-------	-------------

production_event_id	BIGINT IDENTITY(1,1)	PK	Identificador interno del evento.
production_order_id	BIGINT	FK	Orden de producción a la que pertenece.
event_timestamp	DATETIME2	-	Fecha y hora del evento.
produced_quantity	DECIMAL(14,3)	-	Cantidad producida durante ESTE evento.
rejected_quantity	DECIMAL(14,3)	-	Cantidad rechazada durante ESTE evento.

Nota clave: Las cantidades son incrementales por evento. Good Quantity = produced_quantity - rejected_quantity.

10. inventory.inventory_movement

Propósito: Registra todas las entradas y salidas de stock por material y planta.

Granularidad: Una fila por movimiento de inventario.

Columna	Tipo SQL	Clave	Explicación
inventory_movement_id	BIGINT IDENTITY(1,1)	PK	Identificador interno del movimiento.
material_id	INT	FK	Material afectado.
plant_id	INT	FK	Planta donde ocurre el movimiento.
movement_timestamp	DATETIME2	-	Fecha y hora del movimiento.
movement_type	VARCHAR(30)	-	Tipo de entrada o salida.
quantity	DECIMAL(14,3)	-	Cantidad con signo: positiva para entradas y negativa para salidas.

Nota clave: Tipos mínimos: INITIAL_STOCK (+), PRODUCTION_RECEIPT (+), CUSTOMER_SHIPMENT (-). Current Stock = SUM(quantity).

11. Relaciones clave

Origen	Destino	Cardinalidad	FK
master.plant	master.material	1:N	material.plant_id
master.plant	master.production_line	1:N	production_line.plant_id
master.plant	sales.sales_order	1:N	sales_order.plant_id
master.customer	sales.sales_order	1:N	sales_order.customer_id
sales.sales_order	sales.sales_order_line	1:N	sales_order_line.sales_order_id
master.material	sales.sales_order_line	1:N	sales_order_line.material_id
master.material	production.production_order	1:N	production_order.material_id
master.production_line	production.production_order	1:N	production_order.production_line_id
production.production_order	production.production_event	1:N	production_event.production_order_id
master.material	inventory.inventory_movement	1:N	inventory_movement.material_id
master.plant	inventory.inventory_movement	1:N	inventory_movement.plant_id

12. Métricas derivadas principales

Métrica	Cálculo
Pending Quantity	ordered_quantity - delivered_quantity
Order Value	ordered_quantity * unit_price
Good Quantity	produced_quantity - rejected_quantity
Current Stock	SUM(inventory_movement.quantity)
Stock Gap	Current Stock - safety_stock

Rejection Rate	$\text{SUM(rejected_quantity)} / \text{SUM(produced_quantity)}$
----------------	---

13. Lógica de negocio mínima

- El mismo material_code puede estar definido en varias plantas.
- Cada material puede tener un safety_stock diferente por planta.
- La familia del material debe coincidir con la familia de la línea de producción.
- Los production_event guardan cantidades incrementales, no acumuladas.
- Una entrada de producción genera un movimiento positivo PRODUCTION_RECEIPT.
- Una entrega al cliente genera un movimiento negativo CUSTOMER_SHIPMENT.
- El stock actual no se guarda como columna: se calcula sumando inventory_movement.quantity.

14. Orden definitivo de creación

1. **master.plant**
2. **master.customer**
3. **master.material**
4. **master.production_line**
5. **sales.sales_order**
6. **sales.sales_order_line**
7. **production.production_order**
8. **production.production_event**
9. **inventory.inventory_movement**